

IHK AP2 2025 – Theorie-Handbuch (Algorithmen-Kontext)

Plus mit Übungsfragen & Musterantworten

Reiner Theorie-Teil passend zu „Entwicklung & Umsetzung von Algorithmen“. **1 Thema = 1 Seite**, inkl. Übungsfrage & Musterantwort im IHK-Stil.

Inhalt

1. Unit-Tests (Definition, Zweck, AAA)
 2. Whitebox vs. Blackbox
 3. Regressionstests
 4. Abdeckung: Anweisung, Zweig, Pfad
 5. „Wie viele Testfälle nötig?“
 6. Testfalldesign: Äquivalenz & Grenzwerte
 7. Weitere Verfahren (Entscheidungstabellen, Zustände, Pairwise)
 8. Teststufen
 9. Testarten (funktional/nicht-funktional)
 10. Code-Review vs. Test
 11. Testautomatisierung & CI/CD
 12. Rekursion vs. Iteration
 13. Generische Klassen
 14. REST & HTTP (CRUD + Statuscodes)
 15. Fehlerbehandlung & Ausnahmen
 16. Komplexität (Big-O)
 17. Zahlen & Präzision
 18. Testdokumente
-

1) Unit-Tests (Definition, Zweck, AAA)

sehr häufig

Definition: Test einer kleinsten Einheit (Funktion/Methode/Klasse) isoliert vom Rest.

- **Zweck:** Früh Fehler finden, Verhalten dokumentieren, Regressionsschutz.
- **AAA:** Arrange (Vorbereiten), Act (Ausführen), Assert (Prüfen).
- **Eigenschaften:** deterministisch, schnell, unabhängig, wiederholbar.

Übungsfrage

Nennen Sie Zweck und Aufbau (AAA) eines Unittests in einem Satz.

Musterantwort

„Ein Unittest prüft eine einzelne Methode isoliert; mit Arrange-Act-Assert werden Daten vorbereitet, die Methode ausgeführt und das erwartete Ergebnis automatisch geprüft.“

2) Whitebox vs. Blackbox

sehr häufig

Whitebox (kennt Code)

- Fehler präzise lokalisierbar.
- Abdeckung (Statement/Branch) steuerbar.
- Dead Code erkennbar.

Blackbox (nur Ein-/Ausgaben)

- Prüft Anforderungen aus Nutzersicht.
- Technologieunabhängig.
- Eignet sich für externe Komponenten.

Übungsfrage

Nennen Sie je zwei Vorteile von Whitebox- und Blackbox-Tests.

Musterantwort

Whitebox: präzise Fehlerlokalisierung, hohe Abdeckung messbar. Blackbox: Anforderungssicht, unabhängig vom Quellcode.

3) Regressionstests

sehr häufig

Zweck: Sicherstellen, dass nach Änderungen bestehende Funktionen weiterhin korrekt sind.

- Nach Bugfix/Refactoring/Feature/Release ausführen.
- Automatisierte Wiederholung in CI/CD.

Übungsfrage

Formulieren Sie kurz den Zweck eines Regressionstests.

Musterantwort

„Regressionstests stellen nach Änderungen sicher, dass zuvor funktionierende Funktionen weiterhin fehlerfrei arbeiten.“

4) Abdeckung: Anweisung, Zweig, Pfad

häufig

- Statement: jede Anweisung mind. 1×.
- Branch: jeder true/false-Zweig mind. 1×.
- Path: jeder vollständige Pfad (praktisch begrenzen, Schleifen!).

Übungsfrage

Erklären Sie den Unterschied zwischen Anweisungs- und Zweigüberdeckung.

Musterantwort

Anweisungsüberdeckung fordert, dass jede Anweisung einmal ausgeführt wird; Zweigüberdeckung verlangt zusätzlich, dass jede Bedingung sowohl den true- als auch den false-Zweig nimmt.

5) „Wie viele Testfälle nötig?“

häufig

1. Kontrollfluss skizzieren.
2. Für Statement: alle Blöcke einmal erreichen.
3. Für Branch: pro Bedingung true & false abdecken (mit minimalen Kombinationen).

Übungsfrage

Wieviele Fälle sind minimal für Zweigüberdeckung bei einer einfachen if/else-Struktur nötig?

Musterantwort

Mindestens zwei: einer für den true-Zweig, einer für den false-Zweig.

6) Testfalldesign: Äquivalenz & Grenzwerte

sehr häufig

Äquivalenzklassen

Eingabebereich in gültig/ungültig teilen; pro Klasse 1 Fall.

Grenzwertanalyse

min, $\min \pm 1$, max, $\max \pm 1$ testen.

Übungsfrage

Definieren Sie Äquivalenzklassen für Alter 0–120 inkl. Grenzwerten.

Musterantwort

Ungültig: <0 , >120 ; gültig: 0–120. Grenzwerte: -1 , 0, 1, 119, 120, 121.

7) Weitere Verfahren: Entscheidungstabelle, Zustände, Pairwise

manchmal

- Entscheidungstabelle: Regeln aus Kombis von Bedingungen.
- Zustandsbasiert: Zustände/Übergänge testen.
- Pairwise: alle 2er-Kombinationen von Parameterwerten abdecken.

Übungsfrage

Wozu eignet sich eine Entscheidungstabelle?

Musterantwort

Zur systematischen Ableitung vollständiger Testfälle bei mehreren Bedingungen und Regeln.

8) Teststufen

manchmal

- Unit → einzelne Methode
- Integration → Zusammenspiel von Modulen
- System → Gesamtsystem gg. Anforderungen
- Abnahme (UAT) → durch Anwender

Übungsfrage

Was prüft der Integrationstest?

Musterantwort

Die Schnittstellen und das Zusammenspiel mehrerer Einheiten/Module.

9) Testarten (funktional / nicht-funktional)

manchmal

- Funktional: Korrektheit bzgl. fachlicher Anforderungen.
- Nicht-funktional: Performance, Usability, Zuverlässigkeit.

Übungsfrage

Nennen Sie je ein Beispiel für funktionales und nicht-funktionales Testen.

Musterantwort

Funktional: korrektes Ergebnis einer Berechnung; nicht-funktional: Antwortzeit unter Last.

10) Code-Review vs. Test manchmal

- Review: statisch (ohne Ausführung).
- Test: dynamisch (mit Ausführung).
- Beides kombiniert ideal.

Übungsfrage

Unterscheiden Sie Code-Review und (Unit-)Test in einem Satz.

Musterantwort

Ein Review inspiziert den Code statisch, ein Test führt Code aus und prüft das Laufzeitverhalten.

11) Testautomatisierung & CI/CD

manchmal

- CI: automatisches Bauen & Testen bei jedem Commit.
- CD: automatisierte Auslieferung mit Qualitätschecks.

Übungsfrage

Nennen Sie einen Vorteil automatisierter Tests in CI.

Musterantwort

Schnelles Feedback bei Fehlern und dauerhaft stabile Builds.

12) Rekursion vs. Iteration

häufig

- Pro Rekursion: natürlich bei Bäumen/Divide&Conquer.
- Contra: Stack-Overflow, Overhead.
- Iteration: oft effizienter.

Übungsfrage

Nennen Sie je einen Vor- und Nachteil rekursiver Lösungen.

Musterantwort

Vorteil: oft einfacher und näher an der Problemlogik; Nachteil: Stack-Risiko und langsamer durch Aufruf-Overhead.

13) Generische Klassen

häufig

Container/Typen mit Parametern (T) → typsicher & wiederverwendbar.

Übungsfrage

Wozu dienen Generics in Sammlungen (z. B. Liste<T>)?

Musterantwort

Zur Typsicherheit und Wiederverwendung: dieselbe Liste kann Elemente eines konkreten Typs aufnehmen, ohne Casts.

14) REST & HTTP (CRUD + Statuscodes) häufig

- CRUD→HTTP: Create=POST, Read=GET, Update=PUT/PATCH, Delete=DELETE.
- Typische Codes: 200, 201, 204, 400, 401, 404.

Übungsfrage

Ordnen Sie CRUD die passenden HTTP-Methoden zu.

Musterantwort

Create→POST, Read→GET, Update→PUT/PATCH, Delete→DELETE.

15) Fehlerbehandlung & Ausnahmen

manchmal

- Frühe Prüfungen (Guards), klare Rückgaben/Fehlercodes, gezielte Exceptions.
- Negativtests & Grenzwerte einplanen.

Übungsfrage

Nennen Sie eine gute Praxis bei der Fehlerbehandlung in Algorithmen.

Musterantwort

Frühes Validieren von Eingaben (Guard-Clauses) und eindeutige Fehlerrückgaben.

16) Komplexität (Big-O)

selten

- $O(1)$, $O(n)$, $O(n^2)$; binäre Suche $O(\log n)$; Bubble/Insertion $O(n^2)$.

Übungsfrage

Geben Sie die Komplexität der binären Suche an und begründen Sie kurz.

Musterantwort

$O(\log n)$, da der Suchraum in jedem Schritt halbiert wird.

17) Zahlen & Präzision

manchmal

- Integer- vs. Gleitkomma-Division beachten.
- Rundungsregeln festlegen.

Übungsfrage

Welcher typische Fehler kann bei Durchschnittsberechnungen auftreten?

Musterantwort

Ganzzahldivision statt Gleitkommadivision führt zu abgeschnittenen Ergebnissen.

18) Testdokumente manchmal

- Testfall: ID, Vorbedingungen, Schritte, Daten, erwartetes Ergebnis.
- Protokoll: Ergebnis je Fall (bestanden/nicht), Evidenz.
- Fehlerbericht: Reproduktion, Ist/Soll, Schweregrad, Status.

Übungsfrage

Was gehört in einen Testfall mindestens hinein?

Musterantwort

Vorbedingungen, Eingabedaten, Schritte, erwartetes Ergebnis und eine eindeutige ID.

Druck: Datei im Browser öffnen → Drucken → Als PDF speichern. Seitenumbrüche sind eingebaut.